Live Video Streaming System

End-to-end local Wi-Fi live camera streaming built in pure Java using an Android phone and a desktop server.

1. Project Overview

What it does: Captures live camera frames on an Android phone, compresses each frame to JPEG, and streams them over local Wi-Fi via HTTP POST to a Java desktop server running on a laptop for real-time display in a Swing window.

Key Highlights: Pure Java on both ends with no third-party streaming dependencies, no Node.js/Python/FFmpeg required, and a runtime-configurable target IP.

2. System Architecture

The system relies on a direct local network socket link over HTTP POST:

Component	Technology Choice	Why Chosen
Server HTTP	com.sun.net.httpserver.HttpServer	Built into JDK, zero external server dependencies
Server GUI	Java Swing JFrame / JPanel	Native toolkit, zero external dependencies
Image Decoder	javax.imageio.ImageIO	Native Java support for JPEG payload decoding
Android Camera	CameraX (Core, Camera2, Lifecycle, View)	Modern, lifecycle-aware preview and frame analysis
Android HTTP	OkHttp	Battle-tested network client with robust thread handling

3. How It Works — Data Flow

- **Android Preview:** CameraX binds the rear camera to a PreviewView for local monitoring.
- **Frame Analysis:** Frames are sent to an ImageAnalysis analyzer using `STRATEGY_KEEP_ONLY_LATEST` to drop backlogged frames and keep latency minimal.
- **Frame Throttle:** Analyzers check timestamps to enforce ~15 FPS, conserving bandwidth and device battery.
- **JPEG Encode:** RGBA buffers are converted into Bitmaps and compressed to JPEG byte arrays (30–80 KB) at quality 60.
- **HTTP POST:** NetworkStreamer pushes bytes to `http://<laptop-ip>:8080/upload` using an atomic one-in-flight check.
- **Server Handling:** The HttpServer accepts payloads, decodes bytes via `ImageIO.read()`, and dispatches updates to the Swing UI thread safely via `SwingUtilities.invokeLater`.

4. Folder Structure

```
VideoTransmission/ ├── laptop-video-server/ ← MODULE 1 (Desktop Server) | ├── src/main/java/com/stream/server/ | | ├── Main.java (Entry point + IP banner) | | ├── StreamServer.java (HttpServer + POST handler) | | └── VideoWindow.java (Swing JFrame + ImagePanel) | └── run-server.bat | └── mobile-camera-app/ ← MODULE 2 (Android App) ├── build.gradle & settings.gradle └── app/src/main/ ├── AndroidManifest.xml (Permissions & cleartext traffic) ├── res/layout/activity_main.xml (UI controls & PreviewView) └── java/com/stream/app/ ├── MainActivity.java (UI & runtime permissions) ├── CameraManager.java (CameraX setup & frame encoding) └── NetworkStreamer.java (OkHttp transport client)
```

5. Prerequisites & Environment Setup

Ensure required software components are installed prior to compiling and launching the system modules.

Software	Version	Purpose
JDK 26 (or 11+)	Any	Executes the laptop desktop server
JDK 17	17.x	Required build toolchain for Android Gradle Plugin 8.1
Android SDK	Platform 33, Build-Tools 33.0.1	Compiling Android application packages
Gradle Wrapper	8.1.1	Android application build driver

Environment Layout Reference (Windows Example)

Item	Path / Configuration
JDK 26 Default	C:\Users\user\.jdks\openjdk-26.0.1
JDK 17 (Android)	C:\Program Files\Eclipse Adoptium\jdk-17.0.20.101-hotspot
Android SDK	C:\Users\user\AppData\Local\Android\Sdk
Project Root	C:\Users\user\Desktop\Projects\VideoTransmission

One-Time Setup Command:

```
setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-17.0.20.101-hotspot"
```

Restart command prompt shells after execution.

6. Detailed Component Implementation

Laptop Server Architecture

Main.java: Initializes system look and feel, parses port arguments, scans active local network IPv4 adapters, and attaches a shutdown hook for safe resource cleanup.
StreamServer.java: Binds an `HttpServer` to port 8080 using a cached thread pool. Enforces payload limits (4 MB max) and rate-limited logging (once per second).
VideoWindow.java: Implements double-buffered Swing rendering that preserves aspect ratio and scales frames smoothly.

Android Camera Management

CameraManager.java: Configures CameraX with `STRATEGY_KEEP_ONLY_LATEST` and `OUTPUT_IMAGE_FORMAT_RGBA_8888`. Manages stride padding adjustments when generating compressed JPEGs at target quality 60.

Android Network Streaming

NetworkStreamer.java: Uses OkHttp with strict 5-second timeouts. Implements an `AtomicBoolean` flag to enforce a single in-flight network request policy, dropping stale frames during network congestion.

7. Troubleshooting & Quick Reference

Error Message / Issue	Root Cause	Resolution Fix
package com.sun.net.httpserver does not exist	JDK 9+ modular isolation	Add `--add-modules jdk.httpserver`
Unsupported class file major version 70	Gradle executed on JDK 26	Install JDK 17 and configure JAVA_HOME
INSTALL_FAILED_USER_RESTRICTED	OEM USB security block	Enable "Install via USB" in Developer Options
gradlew.bat is not recognized	Wrapper uninitialized	Run `gradle wrapper --gradle-version 8.1.1`

Quick Reference Cheat Sheet

```
# Start Laptop Server cd laptop-video-server javac --add-modules jdk.httpserver -d out src\main\java\com\stream\server\*.java java --add-modules jdk.httpserver -cp out com.stream.server.Main # Rebuild Android APK cd mobile-camera-app set JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-17.0.20.101-hotspot gradlew.bat assembleDebug # Install APK via ADB adb install -r app\build\outputs\apk\debug\app-debug.apk
```